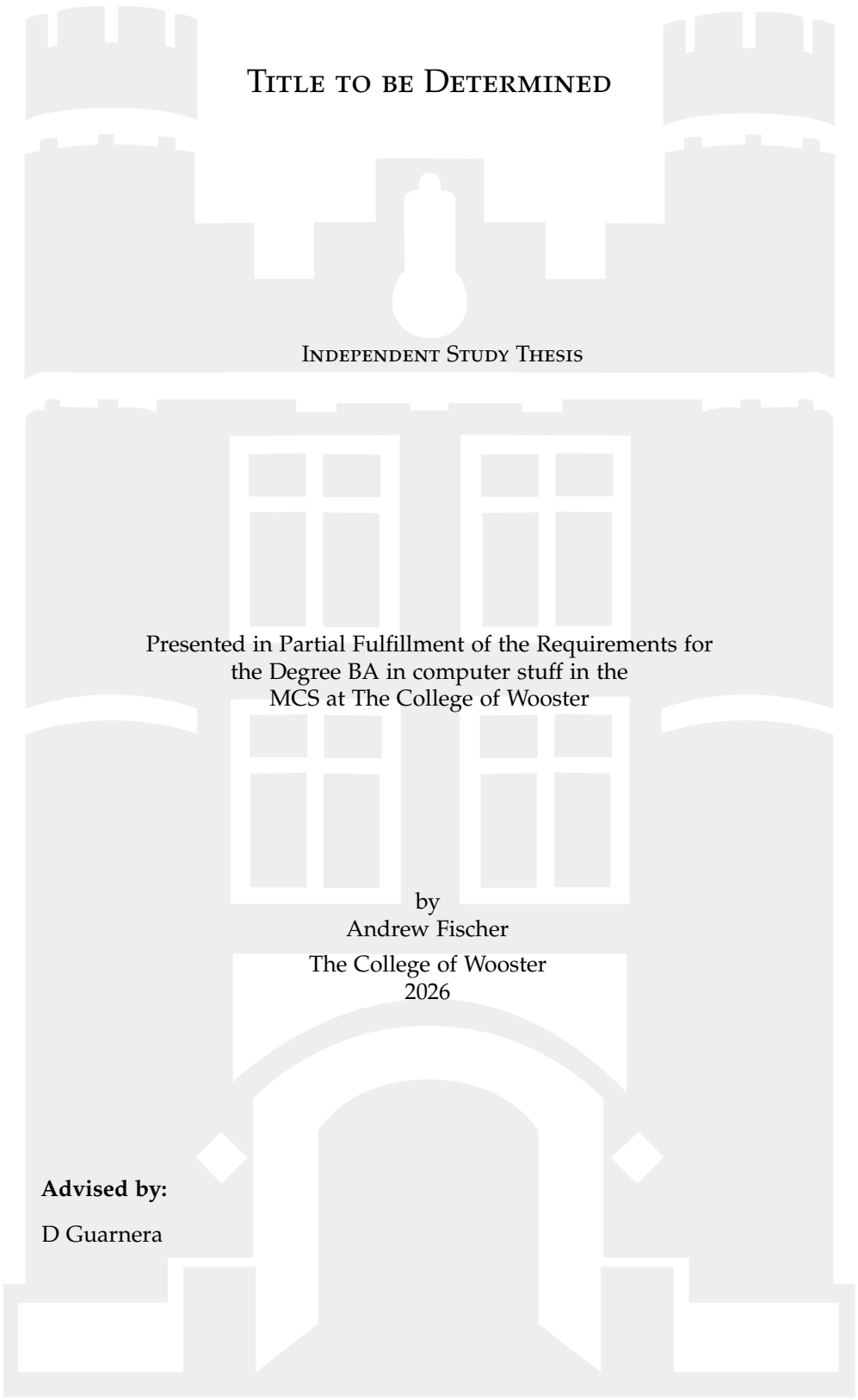




TITLE TO BE DETERMINED

INDEPENDENT STUDY THESIS

Presented in Partial Fulfillment of the Requirements for  
the Degree BA in computer stuff in the  
MCS at The College of Wooster

by  
Andrew Fischer  
The College of Wooster  
2026

**Advised by:**  
D Guarnera



---

THE COLLEGE OF

---

WOOSTER

---

© 2026 by Andrew Fischer  
All rights reserved.

# ABSTRACT

Include a short summary of your thesis, including any pertinent results. This section is *not* optional, and the reader should be able to learn the meat of your thesis by reading this (short) section.

This work is dedicated to the future generations of Wooster students.

## ACKNOWLEDGMENTS

I would like to acknowledge Prof. Lowell Boone in the Physics Department for his suggestions and code.

# VITA

## Publications

Fields of Study Major field: Major

Minor field: Minor

Specialization: Area of IS research

# CONTENTS

|                  |     |
|------------------|-----|
| Abstract         | iii |
| Dedication       | iv  |
| Acknowledgments  | v   |
| Vita             | vi  |
| Contents         | vii |
| List of Figures  | ix  |
| List of Tables   | x   |
| List of Listings | xi  |

| CHAPTER                                      | PAGE |
|----------------------------------------------|------|
| 1 Introduction                               | 1    |
| 1.1 About Last.fm                            | 1    |
| 1.1.1 Artist profiles                        | 2    |
| 1.1.2 Last.fm's "Mainstream Score"           | 2    |
| 2 Background                                 | 4    |
| 2.1 APIs                                     | 4    |
| 2.1.1 Purposes of Web APIs                   | 4    |
| 2.1.2 HTTP and RESTful APIs                  | 5    |
| 2.1.3 Other API structures                   | 6    |
| 2.1.3.1 Event-driven architecture            | 6    |
| 2.1.3.2 HTTP Server-Sent Events              | 6    |
| 2.1.3.3 WebSockets                           | 6    |
| 2.1.4 The Last.fm API and its Limitations    | 7    |
| 2.2 Web scraping                             | 7    |
| 2.2.1 Mechanisms                             | 8    |
| 2.2.2 An example scrape: robertchristgau.com | 8    |
| 2.2.3 Conceptual limitations                 | 9    |
| 2.2.4 Legal and ethical considerations       | 10   |
| 3 Full stack web design                      | 12   |
| 3.1 The Tech Stack in Web Development        | 12   |
| 3.2 Web app functionality                    | 13   |
| 3.3 Single page vs. Multipage                | 13   |
| 3.4 Architecture                             | 14   |
| 3.4.1 Architecture styles                    | 14   |
| 3.4.1.1 Monolithic Architecture              | 14   |
| 3.4.1.2 Microservices Architecture           | 15   |
| 3.4.2 Architecture patterns                  | 15   |
| 3.4.2.1 Layered                              | 15   |

|         |                                                               |    |
|---------|---------------------------------------------------------------|----|
| 3.4.2.2 | Client-Server . . . . .                                       | 16 |
| 3.4.2.3 | Model-View-Controller (MVC) . . . . .                         | 17 |
| 3.4.2.4 | Command and Query Responsibility Segregation (CQRS) . . . . . | 17 |
| 3.4.2.5 | Microfrontends . . . . .                                      | 18 |
| 3.4.2.6 | JAMstack . . . . .                                            | 19 |
| 3.4.2.7 | Flux . . . . .                                                | 19 |
| 3.5     | Containerization and deployment . . . . .                     | 20 |
| 4       | Database theory and design . . . . .                          | 22 |
| 4.1     | Relational databases . . . . .                                | 22 |
| 4.2     | Non-relational databases and NoSQL . . . . .                  | 23 |
| 4.3     | Something about one-file vs distributed nodes . . . . .       | 23 |
| 4.4     | Normalization strategies . . . . .                            | 23 |
| 5       | Data Visualization with the DOM . . . . .                     | 24 |
| 5.1     | The Document Object Model . . . . .                           | 24 |
| 5.2     | Basic principles . . . . .                                    | 24 |
| 5.3     | Scalable Vector Graphics . . . . .                            | 24 |
| 6       | Implementation . . . . .                                      | 26 |
| 6.1     | Goal . . . . .                                                | 26 |
| 6.2     | Last.fm as a data source . . . . .                            | 26 |
| 6.2.1   | Overlapping artists . . . . .                                 | 27 |
| 6.2.2   | Misspelled inputs . . . . .                                   | 27 |
| 6.2.3   | Cleaning the data . . . . .                                   | 28 |
| 6.3     | Processing data with python . . . . .                         | 28 |
| 6.3.1   | Input: reading .txt and .csv files . . . . .                  | 28 |
| 6.3.1.1 | Input via .txt . . . . .                                      | 28 |
| 6.3.1.2 | Input via .csv . . . . .                                      | 28 |
| 6.3.1.3 | Fast mode . . . . .                                           | 28 |
| 6.3.2   | Generating a list of target sites . . . . .                   | 29 |
| 6.3.3   | Scraping: BeautifulSoup and Requests . . . . .                | 29 |
| 6.3.3.1 | Best practices . . . . .                                      | 29 |
| 6.3.4   | Processing: Pandas . . . . .                                  | 30 |
| 6.3.4.1 | Further data analysis . . . . .                               | 30 |
| 6.4     | Design decisions . . . . .                                    | 31 |
| 7       | Conclusion . . . . .                                          | 32 |
|         | References . . . . .                                          | 33 |
|         | Index . . . . .                                               | 35 |

# LIST OF FIGURES

| Figure                                                                                                  | Page |
|---------------------------------------------------------------------------------------------------------|------|
| 1.1 Four scrobbles on a user's profile, each track displaying their album, title, artist, and timestamp | 2    |
| 1.2 An example of Last.fm's graph of artist's six-month listening history . . . . .                     | 2    |
| 1.3 An example of how Last.fm represents its Mainstream Score . . . . .                                 | 3    |
| 2.1 A diagram of a sustained bidirectional connection using WebSockets . . . . .                        | 6    |
| 2.2 An example website's HTML structure . . . . .                                                       | 9    |
| 2.3 Structured data within a webpage . . . . .                                                          | 10   |
| 3.1 An example of layered architecture . . . . .                                                        | 16   |
| 3.2 A simple diagram of the Model-View-Controller architectural pattern . . . . .                       | 17   |
| 3.3 A simple diagram of the Command & Query Relational Segregation architectural pattern .              | 18   |
| 3.4 Contrasting the JAMstack (right) with a traditional layered architecture (left) . . . . .           | 19   |
| 3.5 A simple diagram of the Flux architectural pattern . . . . .                                        | 20   |
| 5.1 An example of a DOM tree . . . . .                                                                  | 25   |
| 6.1 The American Supernova's "Chewbacca" is listed among songs by the Chilean Supernova .               | 27   |

# LIST OF TABLES

Table

Page

# LIST OF LISTINGS

Listing

Page

# CHAPTER 1

## INTRODUCTION

Music has always been a social phenomenon, and that's never been more true than in the internet age, with discussion and discovery being easier than ever before. In the streaming era, the internet has become an important hub for both music listening and discovery, as services have been created not just for people to play music, but to do things like write reviews, catalog their collections, or chronicle their listening histories.

Larger web services like the sales platform Discogs or the listening tracker Last.fm make data from their platforms available to others, allowing an ecosystem of smaller web apps run by independent developers to proliferate using their data, which is for the most part freely accessible. This project will concern the Last.fm service and its associated web app ecosystem.

### 1.1 ABOUT LAST.FM

Last.fm is a web service that collects music listening data through an API. Users connect their Last.fm account to a supported music player like iTunes or Spotify, which sends data about each song played through the software. This data is used to build a comprehensive record of everything they've listened to while connected. The site uses this data to create statistics, visuals, and tools with the data it receives, which will be expanded on throughout this section.

Every song played by a user creates an entry in Last.fm's database. Each contains a song title, artist name, and timestamp, with optional entries for an album name and album artist. They call these entries "scrobbles", after the name given to the software Last.fm runs on, the Audioscrobbler.

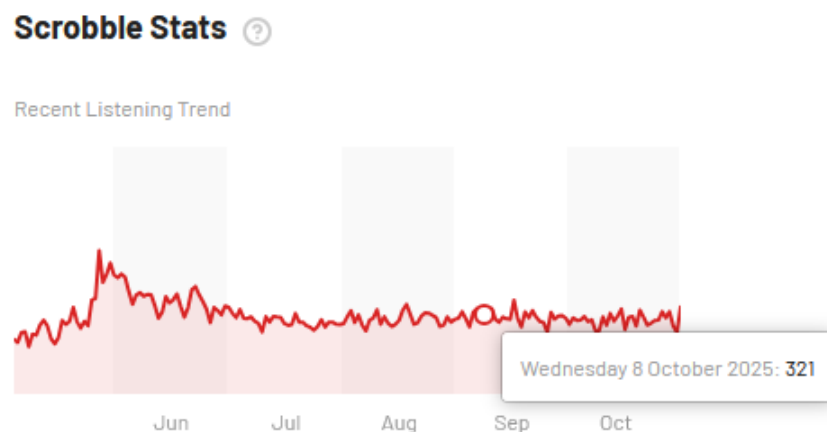
Entries can be deleted by their user at any time, but can only be added within two weeks of the current data and time. This makes data from more than 14 days ago slightly more trustworthy, as potential false or malicious entries can only be created over a two-week period of time.

|                                                                                   |                      |             |                |
|-----------------------------------------------------------------------------------|----------------------|-------------|----------------|
|  | ♡ <b>Straight</b>    | Veruca Salt | 22 minutes ago |
|  | ♥ <b>Get Over It</b> | OK Go       | 25 minutes ago |
|  | ♡ <b>Show Me</b>     | Mint Royale | 29 minutes ago |
|  | ♡ <b>Zombie</b>      | Jamie T     | 29 minutes ago |

**Figure 1.1:** Four scrobbles on a user's profile, each track displaying their album, title, artist, and timestamp

### 1.1.1 ARTIST PROFILES

While the main purpose of Last.fm is to build profiles of users' listening data, the site also builds profiles of each artist, album, and track that gets submitted. Each profile gets its own dedicated webpage that contains an editable wiki, a six-month history of how many unique listeners the artist/album/track had on each day, and other user-generated widgets like a schedule of live performances for artists and a gallery of cover art for albums. These pages are auto-generated the first time an artist/album/track is submitted by a user and automatically refresh their information daily



**Figure 1.2:** An example of Last.fm's graph of artist's six-month listening history

### 1.1.2 LAST.FM'S "MAINSTREAM SCORE"

Last.fm delivers listening reports to its users every week describing their listening trends over that period of time. While all users receive these reports, paying users get access to a few extra stats and charts, including their own calculator of mainstream taste: the Mainstream Score. The tool uses a largely unspecified method

to assign a popularity weight to each artist, and provides the user a normalized score out of 100 and the identity of the most obscure artist they played in that time.

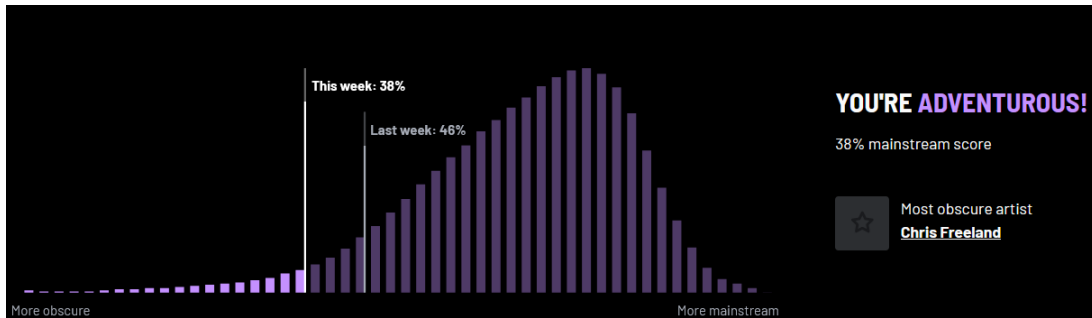


Figure 1.3: An example of how Last.fm represents its Mainstream Score

While this feature provides interesting information to users, it has a few key weaknesses. First, it is only available for paying users, those having a Pro subscription to the site. Second, it offers no further information than the two basic points of information shown in Figure 1.3 – even the histogram displayed is hard-coded and does not change from week to week. Data about the popularity of each individual artist is being collected by the site, but is not available to users in any way.

The aim of this project is to create a web app that uses scraping – a process to remotely obtain information from webpages – to collect publicly available data on the popularity of certain artists and compare it with listening data supplied by a given user. This will improve on the Mainstream-o-Meter in two key ways: first, it will be free and open-source, with the source code uploaded to GitHub for anyone to view or modify as they see fit; second, it provides more information to the user than Last.fm provides, ideally showing the user which artists contributed most to the weighting of their score. The site will take in as input the number of times a user has listened to different artists in a chosen timeframe and will output data about how mainstream the music taste represented by the input data is. A goal of the project is to fit into the wider ecosystem of Last.fm web apps while being the first of its kind to integrate input data from both scraping.

This paper will be laid out as follows: [chapter 2](#) will cover background information on the relevant topics of web scraping and Web APIs [[among others?]]. [chapter 3](#) discusses the standards and practices of how web apps are designed and deployed. [chapter 4](#) discusses the design and structure of databases and how web apps use them for persistent data storage. [chapter 5](#) discusses the data visualization principles behind the visualizations the project will use and the nature of the tools used in designing charts and graphics for web apps. [chapter 6](#) discusses my implementation of the software using the tools and concepts laid out in previous chapters. Finally, [chapter 7](#) concludes the paper.

# CHAPTER 2

## BACKGROUND

### 2.1 WEB APIs

An Application Programming Interface (API) is a connection between two computer systems, as opposed to a connection between a system and a user. The term can be used to describe both how a software makes its output available to external programs, and the ways in which internal software components make information and outputs available to other components. In the context of web and internet services, web APIs accept, handle, or distribute the content of a website or web app. Unlike other, internal APIs, web APIs are usually designed to be exposed over a network and used remotely by multiple users.[3]

#### 2.1.1 PURPOSES OF WEB APIs

In most traditional web pages, any information the site wants conveyed to the user must be taken from the original code and rendered graphically into a form the user can directly view. As web APIs represent computer-to-computer connections, there is no need to create visual elements and data structures that useful to human users. The two main benefits this creates are of automation and accessibility. Without the overhead of rendering graphical elements and with the ease of querying a system programmatically (i.e. via code), receiving and executing requests becomes a more efficient process, making it easier for it to be automated. The availability of data on-demand through an API makes it possible for other sites or services to make of its data, significantly increasing its potential reach and usefulness.

### 2.1.2 HTTP AND RESTFUL APIs

Web APIs often communicate, like traditional webpages, via the Hypertext Transfer Protocol (HTTP), which sets an established protocol for communication over the web. HTTP facilitates a simple system of requests and responses between clients and servers, respectively, rather than a constant stream of data between the two. Requests and responses must each have a particular structure to comply with the protocol. Requests contain the following:

- An HTTP method, most commonly the verbs GET, POST, PUT, or DELETE (which tell the server to fetch, add, update, and remove content, respectively). Other verbs and select nouns like OPTIONS (which asks for specifications to make valid requests to a resource) or HEAD (which functions as a GET request that ignores any returned body content) also exist.
- The path of the resource to fetch; usually taking the form of a URL section like `/api/v1/customers`
- The version of the HTTP protocol
- Headers that may or may not be required by the server to convey additional information
- A body containing the content of a request for certain methods like POST

Responses similarly contain a mix of required and optional components that form a consistent structure. They contain the following:

- A status code, indicating if the request was successful, if so what was accomplished and if not, why not
- A status message, a non-authoritative short description of the status code
- The version of the HTTP protocol they follow
- Optional HTTP headers, like those for requests
- A body containing the content of a response for methods like GET that ask for resources

REST (Representational State Transfer) is a style of API design to make systems that communicate over HTTP using simple, predictable operations. REST-compliant (usually referred to as "RESTful") APIs model resources like files as identifiable URLs, and make all interactions with those resources stateless, meaning each request contains all the information the server needs to process it[7]. A hallmark of REST is closely adhering to the standards of HTTP (e.g. organizing request routes correctly among the GET, POST, PUT, and DELETE methods, despite doing so being arbitrary and optional). By following clear conventions and leveraging HTTP's existing infrastructure, REST makes APIs easier to understand, integrate with, and scale.

### 2.1.3 OTHER API STRUCTURES

Though the structure is extremely popular and often-used, REST is not the only structural option for designing an API. Other architectures exist to cover different use cases and different methods of sending and receiving information.

#### 2.1.3.1 EVENT-DRIVEN ARCHITECTURE

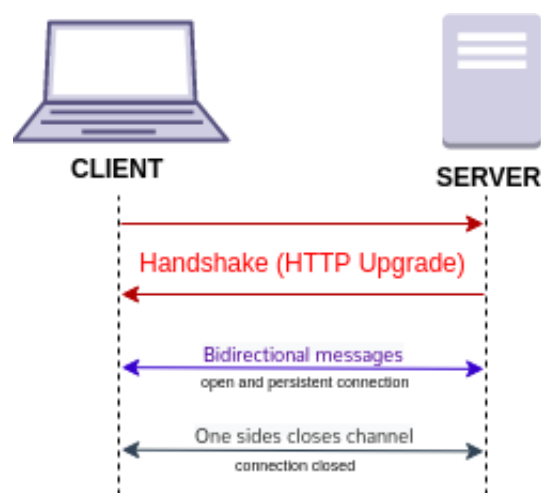
Event-driven architecture (EDA) refers to APIs where the server (or other event provider) sends out messages or signals for clients to receive on open channels. It is, in part, a reversal of the traditional REST structure, as the client waits for a server to send a specific form of information rather than the other way around. This structure is often used on internal APIs

#### 2.1.3.2 HTTP SERVER-SENT EVENTS

Similarly, Server-sent events (SSE) also involve a client listening for data from a server, but often involve a consistent stream of information like stock market figures. This architecture is useful when data is updated often enough that making individual requests would become overly-resource-intensive

#### 2.1.3.3 WEBSOCKETS

Like SSE, WebSockets allow for a constant stream of data, but allow it to flow bidirectionally, meaning textual and binary data is shared in a constant stream between client and server. This method can become very resource-intensive, so care must be taken in what information is allowed to be sent over the connection.



**Figure 2.1:** A diagram of a sustained bidirectional connection using WebSockets

### 2.1.4 THE LAST.FM API AND ITS LIMITATIONS

In addition to receiving listening data from users, the Last.fm API also allows developers to use some of the data Last.fm collects and generates to create their own web tools using listener data. These tools often take the form of websites that create more detailed visuals of users' data than the official site provides. However, there is an important limitation: not all data shown on the site is available through the API. Notably, the six-month listening history for each artist/album/song is absent from the API's methods.

One popular web tool using the Last.fm API is the [Last.fm Mainstream Factor](#) – a site that uses API data to accomplish the same goal of this project: to create a simple numerical score to represent how mainstream a given user's listening history is. However, without access to the six-month daily listener history, the only other data available through the API that can be used to represent an artist's popularity is their amount of all-time listeners – a count of every unique user who has listened to that artist at some point.

The site works by getting each artist's all-time listeners, converting it to a percentage of the largest listener base of any artist (currently [Coldplay](#)), and then averaging the artist's percentage for each track they played in a given timeframe. For example, [Doechii](#) has 15% the amount of listeners as Coldplay, so if you played her songs nine times and a Coldplay song once, you'd end up with a score of 23%, with the larger share of Doechii songs weighing the average toward the bottom end of the scale.

This is a poor method of approximating an artist's popularity, especially over spans of time measuring one year or shorter – the maximum timeframe the Mainstream Factor can measure. It overemphasizes longer-standing artists and one-hit wonders who have many users listen to them at least once, but far fewer that listen regularly. It is also unable to take into account short-term effects on listener-ship such as album releases, where the amount of users listening to an artist will change dramatically over a course of weeks or even days.

To create a better program to determine how mainstream a user's listening history is, we would ideally use the data from the six-month listener history graph contained on their profile page. Since the API does not offer this information, an alternate method of gathering this data must be used. This is where web scraping comes in.

## 2.2 WEB SCRAPING

Web scraping is the process of obtaining the source code of webpages to collect specific data from them, usually using automated software. Whereas an API provides a direct portal where users can request data

from a site's backend, web scrapers request whole pages just as a web browser would, but extract the targeted data from the source code rather than displaying it.

Webpages exist in HyperText Markup Language (HTML), which contains the structure and content of a webpage. When your browser loads a page, it sends an HTTP request to the web address specified by the URL. The server then sends the HTML located at that address to the browser, which renders it to be visible to the viewer. Web scrapers also request a site's code, but instead of rendering it, they keep it in text form and extract what they want from it, be that a data table, word count, list of external links, or whatever a user may want to know.[8] Though other functions like client-side JavaScript provide additional features and interactivity for most pages, for most sites the HTML accessed by a browser or scraper will contain the bulk of a page's content.

### 2.2.1 MECHANISMS

The structured nature of HTML files is very useful to scrapers. With HTML, pages are organized into a hierarchical structure, with elements like paragraphs, lists, data tables and embedded pictures and video contained in marked sections for the browser to properly render them. The classes and tags added to HTML elements for identification by associated style sheet files also come in handy, as a scraper can read these groupings just as easily as a style sheet can. For example, if a sample of text is marked `<span class="highlight">...</span>`, the scraper will be able to identify all highlighted text on a page and separate it from the rest.

An important permutation of this concept is the ability to find the links that a page contains, which are also specially marked in HTML. Keeping a running list of links allows scrapers to traverse to those pages next, essentially traveling on a connected graph of webpages. Programs that perform this function are called "web crawlers" and are used to build sitemaps, collect data from many pages, or simply store all the code it finds to archive the state of each page at the time of retrieval.

### 2.2.2 AN EXAMPLE SCRAPE: ROBERTCHRISTGAU.COM

Figure 2.2 shows the HTML of a simple webpage in which all of the content is arranged in HTML's `<table>` format, which consists of `<tbody>`, `<tr>` (table row), and `<td>` (table data) tags. The figure shows the structure of the outer table, extending to three `<td>` tags. In the case of this webpage – an archive of music critic Robert Christgau's album reviews for a particular artist, The Roots – the first two `<td>` tags contain the page's header and sidebar (each formatted using another nested table, amusingly), while the third contains the reviews, contained in another structure shown in Figure 2.3.

```

<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
    <link rel="stylesheet" href="/rxgau.css" type="text/css">
    <meta name="robots" content="INDEX,FOLLOW">
    <title>Robert Christgau: CG: the roots</title>
  </head>
  <body bgcolor="#e0ffff">
    <table width="100%" border="0" cellspacing="0" cellpadding="10">
      <tbody>
        <tr>
          <td colspan="2" bgcolor="#1010e0">
        </tr>
        <tr>
          <td bgcolor="#1010e0" valign="top">
          <td valign="top" width="100%">
        </tr>
      </tbody>
    </table>
  </body>
</html>

```

Figure 2.2: An example website's HTML structure

Unlike the other two outer `<td>` tags, this section does not encapsulate its data in a `<table>` tag. After a few `<h2>` (heading) and `<ul>` (non-enumerated list) tags to display the page info, the page separates each entry into `<p>` (paragraph) tags, which by default display the text within each of them in the order they appear in the HTML document. The `<p>` tags contain more tags delineating the name of the album, the label and release year, the text of the review, and the rating (two asterisks for the displayed entry, indicating an above-average grade on Christgau's esoteric scale).

On this website, each page of reviews is structured in the same way, with three tables and the third one containing the page's main content. This gives a scraper some flexibility in how it would search for and access any information it wanted: it could find all the tables in the document and find the content table by looking for a table that contains the specific combination of header and paragraph tags, or simply count to the third table on the page and access it, as all pages' third tables will be the content tables. This example shows how the structure of a simple HTML file is seen from the perspective of a scraper.

### 2.2.3 CONCEPTUAL LIMITATIONS

If a scraper is designed to retrieve specific information from a webpage it visits, it often depends on the structure of the HTML to be able to find it quickly and easily. If a site's format and page structure is inconsistent across multiple pages, or if the changes altogether, the scraper will break. Clever use of error

```

<td valign="top" width="100%">
  <!--end standard header-->
  <h2>The Roots</h2>
  <ul> </ul>
  <h3>See Also:</h3>
  <ul> </ul>
  <h3>Consumer Guide Reviews:</h3>
  <p> </p>
  <p> </p>
  <p> </p>
  <p>
    <b>
      <i>The Roots Come Alive: Open Access</i>
    </b>
    [MCA, 1999]
    <br>
    World-class DJ and beatbox, excellent drummer and bassist, pretty darn good rapper(s), bourgie jazzmatazz ("Proceed," "Love of My Life").
    <b>**</b>
  </p>
  <p> </p>
  <p> </p>
  <p> </p>
  <p> </p>
  <p> </p>
  <p> </p>
  <p> </p>
  <p> </p>
  <h3>See Also</h3>
  <ul> </ul>
  <!--begin standard footer-->
</td>

```

**Figure 2.3:** Structured data within a webpage

handling or fallback conditions can mitigate these issues, but scrapers are, by nature, largely at the mercy of whatever site they are accessing.

Because web scraping accesses webpages using the same channels a normal user opening a webpage would, some discrepancies between the two systems may cause problems. Unlike web APIs, which are designed to be accessed programmatically and not directly by users, scraping automates and streamlines interactions with a system designed for human interaction. This could lead to a scraper being identified as a malicious actor (falsely or not), and introduce ethical challenges that are detailed in the next section.

## 2.2.4 LEGAL AND ETHICAL CONSIDERATIONS

Because websites are typically designed for users to access and view them in a web browser, two main ethical concerns can arise from loading large amounts of web pages to gather data:

First, there are concerns of server load. Scrapers request webpages far faster than humans can usually read them, so requesting many pages from the same site will lead to your scraper demanding an outsized share of the processing power a site can provide. Second, some site owners might not want data contained on their site to be collected and stored, even if on the surface it is accessible. In the early days of web development, these concerns led to the creation of a web design standard: robots.txt[8].

robots.txt is a simple text file that most websites will store in their root directory (i.e., the robots.txt file for example.com could be found at example.com/robots.txt). Its purpose is to tell web scrapers which pages on the site, if any, they are allowed to access. Some sites don't care and leave the file empty, some are very restrictive and only allow access to search engines like Google, others operate somewhere in between. In legal cases involving unwanted web scraping, courts have generally interpreted the lack of a restriction in a site's robots.txt file as consent for a web scraper to access that part of the site[8]. While workarounds certainly exist, many popular consumer web scraping tools will not allow you to access sites restricted by a robots.txt file.

Last.fm's robots.txt file prohibits scrapers from accessing users' listening history and data, but does *not* prohibit access to the profile pages where the charts we want to access are stored. This means that the project is in the clear functionally, ethically, and legally until further notice.

# CHAPTER 3

## FULL STACK WEB DESIGN

This section will discuss some shit idk.

This section will introduce the concept of the tech stack and discuss concepts of full-stack web development and their relevance to this project's goals. Starting with a primer on the basics of web applications and information transfer over the internet and following with sections on conceptual differences between single- and multi-page applications, concepts and examples in the software architecture of web apps, and concluding with an overview of the deployment process and the role of containerization.

### 3.1 THE TECH STACK IN WEB DEVELOPMENT

In the field of computer programming, a software stack is the set of software components required to run and support an application[1]. The stacks one uses for different applications can vary, but popular combinations of components have proliferated over time due to the compatibility of certain components with one another. In web development, a common delineation drawn between stack components is between the "frontend", which encompasses the presentation and organization of a site's content, determining how a site is made to appear to a user; and the "backend", which encompasses the logic of the site, processing user requests by routing them to the right component and managing application operations.

An example of a popular web development stack is MERN, an acronym standing for the four components it contains:

- **MongoDB**, A database management system to store information needed by the application. It is non-relational, a concept explored in [chapter 4](#).
- **Express.js**, a framework for designing web applications with JavaScript
- **Angular**, a framework for designing user-interfaces

- **Node.js**, a runtime environment allowing JavaScript to be executed outside of a browser.

Each component in the stack fulfills a different purpose, and together are able to run a web application. Since MERN does not require a specific operating system to run within [\[Cite a MERN thing!\]](#), it is known as an "OS-agnostic" stack. An example of the opposite – an "OS-level" stack – is LAMP, which requires use of the Linux operating system. The rest of the LAMP acronym stands for Apache, the framework to run the backend server; MySQL, the database manager; and Perl, as the primary coding language used.

While some aspects of the MERN and LAMP stacks align with the other (e.g. both specify a database manager and a web development framework), they are not universal among web development stacks. Types of component that exist in one stack might be redundant or unnecessary in another. [\[connect better to next section, whatever it'll be\]](#)

## 3.2 WEB APP FUNCTIONALITY

A key concept in the discussion of web apps and services is the Hypertext Transfer Protocol (HTTP), the Internet's basic communication protocol. It operates on a simple model of requests and responses: a client—usually a browser—sends an HTTP request to a server, specifying a method such as GET, POST, PUT, or DELETE, along with headers and optional body data. The server processes that request and returns an HTTP response, which includes a status code, headers, and usually a body containing the HTML code for the web service being accessed. HTTP is stateless, meaning each request is independent unless additional mechanisms like cookies, sessions, or tokens are used to maintain continuity [\[9\]](#).

In web design, HTTP forms the boundary between the frontend and backend. Browsers, mobile apps, or other clients communicate with backend servers through HTTP to retrieve data or trigger server-side actions. RESTful API designs map HTTP methods to specific actions or resources, while single-page applications rely on code sent to and stored on the client-side to update the UI dynamically. HTTP defines how components across the web exchange information through well-defined request/response patterns.

## 3.3 SINGLE PAGE VS. MULTIPAGE

One key design decision crucial in the early stages of creating web applications is the choice between creating a single-page or multi-page application. The distinction is fairly intuitive: single-page applications load only one HTML page initially and use client-side code like JavaScript to perform dynamic functionality such as fetching data and changing the user-interface. Multi-page applications, on the other hand, use more than one HTML page[\[1\]](#). While the actual size of the sitemap in these applications can vary widely –

anywhere from a small handful of pages to dozens – it is seen as a significant distinction to go from one page to multiple, no matter the exact level of the difference.

The decision whether to structure an application as single-page or multi-page depends on how a designer wants information to be organized. Amazon is a multi-page application where information about each product is contained on a separate page. Google Maps, on the other hand, mostly operates on one homepage that loads the base map pulls up overlays depending on what locations the user searches for or selects.

## 3.4 ARCHITECTURE

The tech stacks in modern software and development often comprise multiple components that interact to perform its functionality. As we saw earlier, different stacks can contain or emphasize different types of components while being agnostic on others. The structure of these components and how they connect with one another can be seen as a basic definition of software architecture. Depending on the needs of the software for its stakeholders, it may need a different combination of components, or commonly-used components assembled and connected in a different way [4]. In web development, while the frontend and backend are commonly-used categorizations of services and components, there are many ways to structure an application, some blurring the lines between the categories.

This section will cover both high-level and low-level architectural concepts with a primer on the basics of web applications and the transfer of information over the internet.

### 3.4.1 ARCHITECTURE STYLES

An architecture style represents a combination of design decisions and principles to organize an application in a certain way. It is the highest level of organization, not used to solve a problem but to enhance certain good practices in a particular type of system.[4]

#### 3.4.1.1 MONOLITHIC ARCHITECTURE

Monolithic architecture is a software development model where all components are interdependent and interconnected; every component is required to run the application. It treats the entire code as a single program, building a new project, applying frameworks, scripts, and templates, which makes testing easier.

The main drawback to this style of architecture is that it becomes difficult to manage or update the code

base as it grows. As each element is interdependent, scaling the application is complex. Introducing even a single point of failure can break the application.

Monolithic architecture is best used for simple applications that have any future scalability as pre-planned as possible. Knowing as much about the future scope of a project's features in advance as possible helps determine what will be feasible or not under a given structure.[11]

#### 3.4.1.2 MICROSERVICES ARCHITECTURE

Microservices architecture serves as the logical counterpoint to a monolithic environment. Using it, code is organized among independent services that don't rely on one another to perform their specific function and communicate via RESTful APIs. Each microservice contains its own database and operates under its own business logic. Because it's loosely coupled, microservice architecture makes each individual service much easier to scale while maintaining continuous delivery. Developers can quickly adapt to innovation.

However, deploying multiple services comes with drawbacks. When the number of services grows, the complexity of managing them grows, too. Microservices architecture is a good choice for highly complex applications with the resources to maintain and scale multiple independent services.[11]

### 3.4.2 ARCHITECTURE PATTERNS

In contrast to styles, architectural patterns are decisions and principles intended to avoid specific issues with the low-level implementation of the software. Essentially, the style comes first and determines the project's broadest level of structure, and the patterns determine the specifics of how each of the components interact with each other[4]. Below are the most prominent architectural styles and how each covers different possible use cases with their strengths and weaknesses.

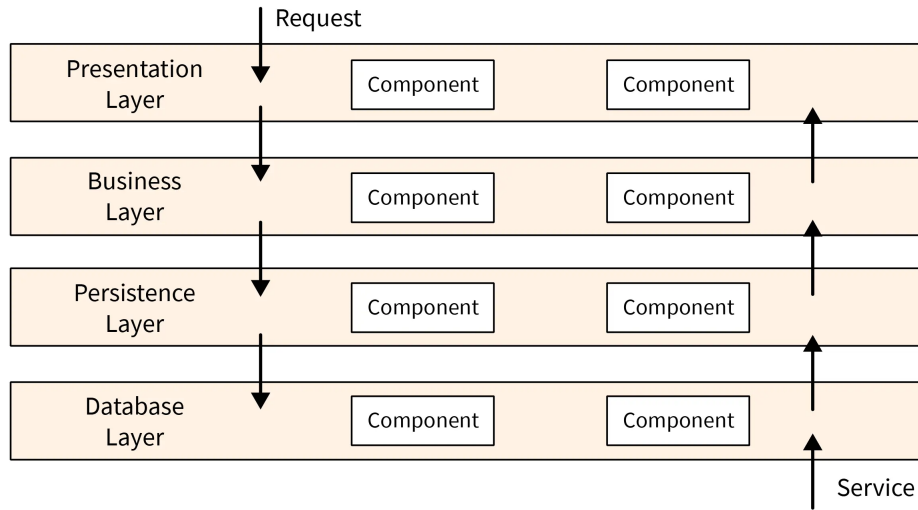
[Maybe go back later and put the book's diagrams in]

#### 3.4.2.1 LAYERED

The layered pattern is a simple way to organize services in a web application. In it, information simply flows through different levels of the software to and from the user and the system in sequence, interacting with the code at each level before being passed on to the next one. The most common permutation of this pattern comprises four layers:

- presentation (or UI), consisting of everything the user sees and interacts with. Pages, menus, text, images, etc.
- business, in which the business logic of the application is defined and operated

- persistence, containing mapping functions and other operations
- database, where information is persistently stored



**Figure 3.1:** An example of layered architecture

While the simplicity of this pattern makes it easy to understand and useful for lightweight, small-scale applications, the lack of flexibility brought on by sending information through many consecutive layers can be inefficient for large applications. Since the layers are interdependent, it also introduces many potential points of failure should one or more layers develop a fault.

#### 3.4.2.2 CLIENT-SERVER

This pattern delineates just two types of process: clients that need information and send structured requests for it, and servers that carry out these requests by fetching and processing the information, thus serving the clients. Often communicating through APIs, it is the server's job to stay open and available for requests from clients.

Even simpler than the layered pattern, client-server promises significant flexibility and ease-of-use. Good security is a must, however, and its main drawback is that the single-route connection can become a performance bottleneck if not properly addressed. Though all websites are, in a sense, built for client-server relationships in the form of user-to-service interaction, this pattern illustrates an architecture where there are only two components interacting at all – distilling the relationship down to its most basic possible extreme.

### 3.4.2.3 MODEL-VIEW-CONTROLLER (MVC)

Put simply, the Model-View-Controller pattern separates the components into three roles: backend logic, frontend presentation, and an intermediary to connect the two, respectively.

- **Model** – contains business logic, core functionality, and persistent data storage. It does not contain any logic that shapes how the data is presented to the user; that is the controller's job
- **View** – Displays information to and interacts with the user. Takes data from the Model and Controller, processes it, and dynamically renders it for a user.
- **Controller** – Processes the inputs the view receives from the user to send to the model and returns data to the view and the user from the model.

This pattern is less straightforward than the others, which can make it unsuitable for some projects that aren't prepared to handle its particular style of decoupling. This pattern illustrates how multiple functions can be performed by a single layer in such a way that the flow of data is not interrupted. This manifests in how the controller handles the input/output of both the view and model, and how the model acts both processing and data storage. [[be clear about what makes this different from Layered, because now it just seems like different layers]]

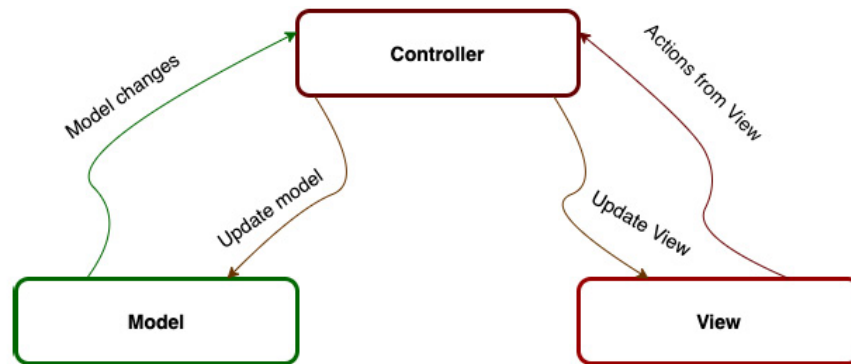
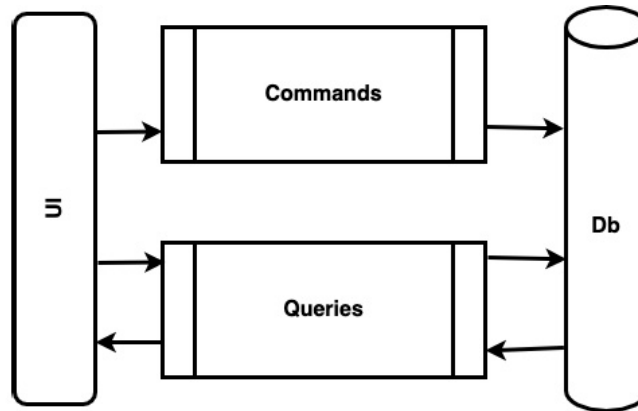


Figure 3.2: A simple diagram of the Model-View-Controller architectural pattern

### 3.4.2.4 COMMAND AND QUERY RESPONSIBILITY SEGREGATION (CQRS)

The signature aspect of the CQRS pattern is its separation of data operations into reading- and writing-based actions: queries and commands. Queries are used to gather data from the database storage; the user specifies what data they want and the application fetches it for them. Commands are responsible for changing data, and can carry out a wide range of actions on it – usually creating new data, reading present data, updating existing data, and deleting data (a set of basic commands commonly known as CRUD).

The advantage of CQRS that it separates the intermediate layer of the application into two wholly distinct components that can operate independently – this way more attention or resources can easily be given to one or the other if a particular project is more reading- or writing-based. It should not, however, be seen as a simple default for database-centric applications; if all you need in an application is the basic CRUD commands, CQRS is likely far more technically complex than is necessary. Usually working with GraphQL, CQRS is a pattern that is geared toward working with query languages to fetch and manipulate data.



**Figure 3.3:** A simple diagram of the Command & Query Relational Segregation architectural pattern

#### 3.4.2.5 MICROFRONTENDS

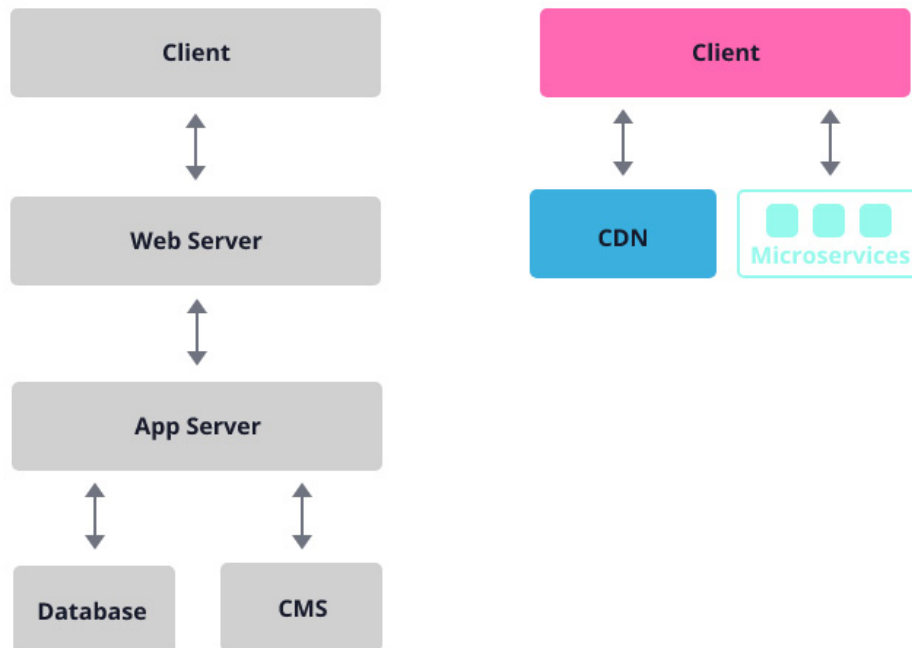
Where the user-facing functionality of an application is contained in a single layer in many other patterns, microfrontends refer to applications that split the user interface (UI) into multiple services, usually worked on by different teams, each with different tech stacks. The pattern essentially extends the microservices style to the frontend, greatly enhancing the capability of the UI by breaking up the monolith. Disadvantages of microfrontends include potential conflict between differently configured services and the difficulty of testing the frontend from the perspective of any one team.

The emergence over time of the microfrontend pattern exemplifies the growing importance and complexity of user interfaces in the modern web. While naturally better-suited to large development teams and resource-intensive frontends, microfrontends serve as a proof-of-concept that it's just as possible to prioritize and expand the functionality of the user interface as with any other segment of an application.

### 3.4.2.6 JAMstack

Where microfrontends use multiple services to make the user interface more layered and complex, JAMstack (JAM stands for "Java, API, and Markup") moves in the opposite direction. The main principal of JAMstack is the lack of a traditional backend server – pages are instead sent to the user through a Content Delivery Network (CDN), which can send files to the user but does not hold data or process requests. To accommodate this, rather than use dynamic webpages as all previous patterns generally do, JAMstack instead relies solely on static, pre-rendered webpages written in HTML and JavaScript. Without a web server, the client-side JavaScript code is responsible for handling communication with the CDN and any other microservices, such as data storage or identity management.

Unlike patterns focused on optimizing for the dynamism of their frontend and/or the scalability of their backend features, JAMstack optimizes for space and cost management by eschewing dynamic frontends and many common backend features entirely. This pattern is most useful for simple web tools and other applications without the need to increase in scale over time.



**Figure 3.4:** Contrasting the JAMstack (right) with a traditional layered architecture (left)

### 3.4.2.7 FLUX

Flux is a pattern created by Facebook, designed to work with the service's React frontend components to build client-side web applications. It was created as an alternative to the Model-View-Controller pattern

described earlier to avoid unnecessarily tangled connections between the services in an application [10]. Flux has a unidirectional data flow, meaning data is always sent through the services in the same sequential order. In the most basic example, the app has three services that process an action taken by a user:

- **Dispatcher** – Receives the action taken by the user and determines both where to send it, and the order information is sent in
- **Store** – Contains the business logic, which it operates on data received from the dispatcher. Also manages the state of the application
- **View** – Receives specifications from the store and renders the information to be presented to the user. The view re-renders when the store announces changes and is responsible for creating new actions based on user input and interaction

Each of these services only sends data in one direction and receives it from the other without a need to double back to confirm or update data. This results in a simpler application that is easier to follow and debug while each service can be optimized for its narrow focus.

Flux was designed with a specific use case in mind: managing React components simply and efficiently. Its lack of two-way inter-service communication would not be a good fit for a project managing large amounts of information, but for a user interface-focused application that doesn't send data more complex than basic text and/or images, it makes perfect sense.

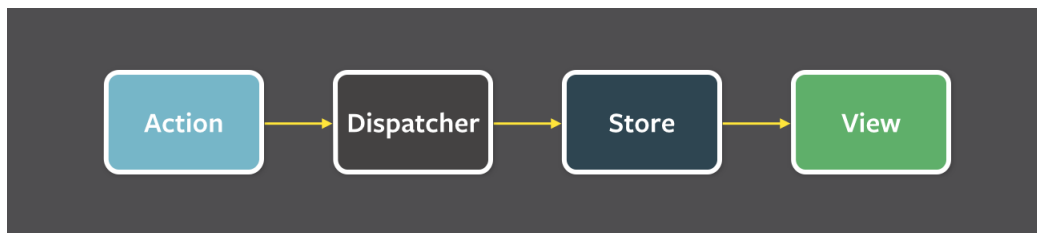


Figure 3.5: A simple diagram of the Flux architectural pattern

## 3.5 CONTAINERIZATION AND DEPLOYMENT

When applications move from the development to the deployment stage, the components of an application must have concrete ways of interacting with each other and the hardware (usually servers) required to become accessible to end users. A popular solution in recent times is containerization[1].

Containerization is a way of packaging an application together with everything it needs to run, like its code, system libraries, and configurations, into a single unit that can run on a physical or a virtual machine

(VM). By existing at the OS level, containerization allows a developer to run multiple containers inside a single OS instance. While it reduces overhead and processing power, it also makes them much faster to start and more resource-efficient.

Tools like Docker build these containers from reproducible definitions (which they call Dockerfiles), ensuring the application runs the same way on different machines, maintaining consistency in development, testing, and deployment. Containerization tools allow teams to ship applications as portable, self-contained artifacts that can run reliably across different environments such as local servers or cloud platforms.

# CHAPTER 4

## DATABASE THEORY AND DESIGN

A database is often defined as an organized collection of information used to model some type of organizational process. In software, databases are the go-to way to store information persistently – meaning data is saved in a location that will not be refreshed after the program concludes are refreshed. In web development, databases are located on the backend and store information that is accessed by the server to process and send, in some form, to a client[6]. This chapter covers various permutations and architectures of databases in the realm of web development.

### 4.1 RELATIONAL DATABASES

The most common form of database is the relational database. Invented in and popularized since the late 1960s, these databases store information in "relations", which are commonly known and seen as tables and represent a specific entity such as customers, orders, or products. Within each table, each row represents a unique record of that entity, and each column defines an attribute of the record.

Every table includes a primary key: a field unique to its table that ensures each row can be referenced unambiguously. Relationships between tables are established through foreign keys, which associate one table's records to those in another via the first table's primary key. This structure allows data to be stored in an organized form that reduces redundancy and maintains consistency.

Functionally, relational databases use Structured Query Language (SQL) to define, manipulate, and query data. SQL is used to insert, update, delete, and retrieve information in a database. The relational model supports strong data integrity, which makes its databases widely used in applications where accuracy is essential.

## 4.2 NON-RELATIONAL DATABASES AND NoSQL

Relational databases offer a number of advantages, but some contexts do not require all of them and would benefit from a system of database design exchanging unnecessary features for increased resource efficiency and reduced overhead. Non-relational databases (often called NoSQL to emphasize the lack of support for SQL queries relational databases use) lack the schemas relational databases use to define relationships and are often used to handle semi-structured data that needs more flexibility to store [2].

## 4.3 SOMETHING ABOUT ONE-FILE VS DISTRIBUTED NODES

## 4.4 NORMALIZATION STRATEGIES

# CHAPTER 5

## DATA VISUALIZATION WITH THE DOM

Once data about each of the user's listened artists has been collected by the scraper and processed by the backend logic, the data then has to be shown to the user. Creating visualizations directly on the webpage (as opposed to creating a graphic separately) involves interacting heavily with the Document Object Model (DOM) – an interface that structures the layout of webpages. This section will discuss the function and scope of interacting with the DOM, and the principles of data visualization that go into representing the data we obtain from the user's listening history.

### 5.1 THE DOCUMENT OBJECT MODEL

The DOM is an interface that represents the structure of a webpage as a hierarchical tree of objects (an example is seen below in figure XX). When a browser loads an HTML document, it converts each element (such as paragraph or image blocks) into nodes in this tree. This structured representation allows programming languages like JavaScript to traverse and modify the content and layout of a webpage. Because the DOM is able to be updated after its initial rendering, it can be coded to respond to user interactions like clicks or keystrokes.

[[explain stuff about how the DOM affects visual elements]]

### 5.2 BASIC PRINCIPLES

### 5.3 SCALABLE VECTOR GRAPHICS

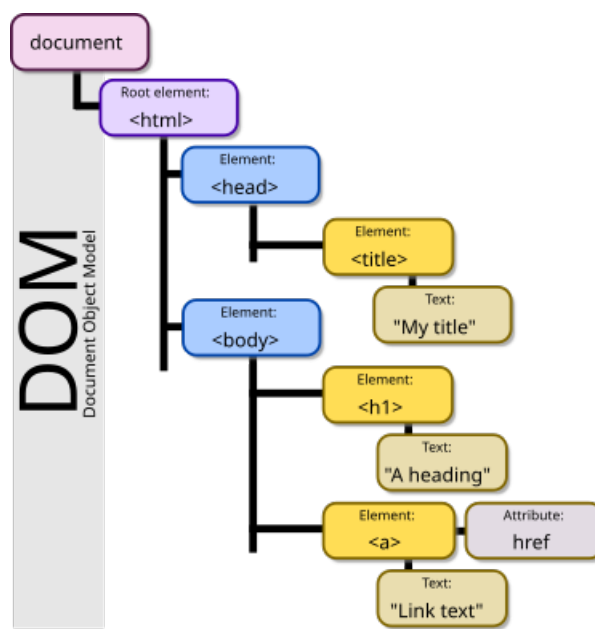


Figure 5.1: An example of a DOM tree

# CHAPTER 6

## IMPLEMENTATION

This section will discuss the application of the topics discussed in creating the final software component.

### 6.1 GOAL

The end goal of this project is to create a web application that, given a user's music listening history as input, scrapes listener data from Last.fm artist pages and presents to the user data about listener trends of the artists in their history.

### 6.2 LAST.FM AS A DATA SOURCE

The nature of the Last.fm database differs from similar services like Spotify's API and necessitates an understanding of how its data is organized and stored to make informed decisions on how to process it and interpret the results.

The most important aspect of Last.fm's data collection and cataloging is that its information is based solely on the literal text of its inputs. Music programs people use to send data to the site send just five pieces of information: artist name, song title, and timestamp – which are all required to create an entry of a played song – plus album title and album artist, which are optional. All of these (with two exceptions we will address later [redirects and the 14-day window]) are stripped of their upper- or lower-casing, then interpreted directly by Last.fm and processed into the database as-is, containing no more data than the strings they were sent in as.

This approach differs from other databases like Spotify's collection of artist and listener data or MusicBrainz' publicly editable user-run database because it does not assign a separate ID number to artists,

albums, or tracks, instead using the text itself as the primary identifier. The approach introduces certain benefits, drawbacks, and other considerations that will be discussed in the following sections.

### 6.2.1 OVERLAPPING ARTISTS

Because the text of an artist's name is used as a unique identifier, the obvious issue this introduces is that artists that share the same name are indistinguishable from one another. Take Supernova as an example: on one hand it is the name of a Chilean girl group active in the early 2000s who scored a few hit singles in their home country. On another, it is the name of an obscure American 1990s punk band best-known for a novelty single about Chewbacca that was featured on the soundtrack to *Clerks* (1999). Both artists share the same space in the Last.fm system, including the artist page, meaning if you look at the complete list of Supernova's played tracks (shown in figure 6.1), both bands' songs will be in the list.



**Figure 6.1:** The American Supernova's "Chewbacca" is listed among songs by the Chilean Supernova

This presents a problem for our data collection because the Chilean group, while by no means a hugely popular artist today, still commands a greater listenership on the whole than the American group, despite the latter group's one successful song. This means that using Last.fm data to determine the popularity of the American Supernova is inherently inaccurate, as the data would be presenting the combined listenerships of both, and would say they are far more popular than they actually are.

### 6.2.2 MISSPELLED INPUTS

Another potential issue with the data Last.fm retains of a user's listening history is that since the inputs come directly from software the user uses, it naturally can contain clerical errors like misspellings. This brings us to the first form of quality-control Last.fm performs on its database: artist redirects. Many artists

in Last.fm's database have automatic redirects set up to catch common misspellings and misinputs – for example, "Brian Transeau" will redirect to his proper stage name "BT", and "The Beetles" will redirect to "The Beatles".

### 6.2.3 CLEANING THE DATA

## 6.3 PROCESSING DATA WITH PYTHON

This section will discuss how the above utilities are implemented in the working program.

### 6.3.1 INPUT: READING .TXT AND .CSV FILES

Upon running the software, the user will be given two choices as to how they can input their desired listening data. First, they can enter it manually, in-browser. Second, they can download a .csv file from [a web tool](#) that exports a Last.fm account's listening data. This subsection describes how each input is handled by the program.

#### 6.3.1.1 INPUT VIA .TXT

[[My plan is to let users type in their history manually if they don't have an accessible history]]

#### 6.3.1.2 INPUT VIA .CSV

For the .csv option, the user is given instructions to go to an external site to generate a .csv of their listening history. This site uses Last.fm's API to gather every song entry from the current date and time back to a time specified by the user. Since the six-month listener graph is shown on a three day delay, we need to select a date ten days in the past to get a week's worth of data from it. As such, the software generates the timestamp of ten days in the past for the user to enter into the website.

[[The user will then upload this file to the site]] The program then iterates through every entry and keeps track of how many times each artist appears in a separate dictionary. It then appends each artist in the dictionary to aligned lists so the output of either input method is the same.

#### 6.3.1.3 FAST MODE

Because scraping a site takes so much more time than other Python processes, the number of artists appearing in one's listening history will have a significant effect on the runtime of the program. To make the program more convenient for users who listen to many artists just once, the program gives them an

option to exclude artists with only 1 recorded entry. If selected, the program skips any artist with only one entry when appending the artists and playcounts to their respective lists.

### 6.3.2 GENERATING A LIST OF TARGET SITES

Since Last.fm profile pages take the form `https://www.last.fm/music/[artist name]`, many artists' names can simply be concatenated to the preceding part of the link. However, if the artist's name contains spaces or other select punctuation, those characters need to be replaced in order to be a valid url. Spaces are replaced by pluses, and other characters like question marks are replaced with unicode designations like `"%3F"`. For example, the name "Chance the Rapper" produces the link `"https://www.last.fm/music/Chance+the+Rapper"`, and the name "+44" produces the link `"https://www.last.fm/music/%252B44"`. It then stores the links in another list aligned with the artist name and playcount.

### 6.3.3 SCRAPING: BEAUTIFULSOUP AND REQUESTS

When a list of valid links has been obtained, the software iterates through it, using Requests to get the source code for each profile page and using BeautifulSoup to turn the response into an object containing all the site's code in text form.

Once each page's code has been obtained, the program uses BeautifulSoup's functions to search for tables within it. As the site is structured now, the second table on the page contains the six-month listening history. Web scraping projects like this often run the risk of breaking if anything on the site changes, and this one is no exception. If the table we're using changes or disappears, there isn't a way to fix it without going back in and re-coding the program. Thankfully, the site's layout has remained the same for just under six years as of this writing, so the likelihood of constant fixes being required is low.

Once the table is acquired, each row is searched for its `"js-date"` and `"js-value"` keys, which are themselves searched for `"datetime"` and `"data-value"` values, respectively. Each row is parsed in this way, and the results are transformed from lists to a Pandas dataframe, which the next section will detail.

#### 6.3.3.1 BEST PRACTICES

One of the main roadblocks to web scraping is that many websites do not want to be scraped and will implement design choices to block or hinder potential scrapers. As discussed previously, Last.fm does not disallow the automated access of its artist pages, so it can be assumed that they are not hostile to scrapers

in general. Still, it's important to not abuse this potential privilege and remain in the good graces of the administrators.

Existing literature recommends setting a minimum delay between requests and testing the limits of your access through trial and error [8]. A delay of 3 seconds was used during the testing of the software, and no restrictions were detected. This renders any potential methods of disguising the identity of the scraper unnecessary for the time being, though they will be described further in Future Work.

#### 6.3.4 PROCESSING: PANDAS

The table extracted by BeautifulSoup is then transformed into a Dataframe by the Pandas module, which takes from each artist the listener counts from every date and calculates the average over four timeframes: one week, one month (30 days), three months (90 days), and six months (180 days; the maximum stored by the table). These averages are appended to the Dataframe, attached to each artist and their playcount. Currently, the software only makes use of the one week timeframe, as it takes the shortest time to run. The implementation of the other timeframes will be discussed in Future Work.

To weight the listener averages, first a new column is created in the Dataframe multiplying each artist's playcount by their average listeners over the one week timeframe. The sum of all values in this new "weighted" column is then divided by the sum of all values in the playcount column, yielding a final value of the user's listener average over the timeframe. The significance of this value is: for each song the user listened to in the last week/month/etc., how popular on average was the artist, in terms of average daily listeners over the same time period.

For most users, this number ranges from 5,000 to 40,000. However on its own the number means very little. After displaying it, the program then performs some additional operations to provide the user with context for their score.

##### 6.3.4.1 FURTHER DATA ANALYSIS

In addition to the basic processing, more context is provided about the artists within the user's dataset. Specifically, which artists were most and least mainstream, and which ones affected the final result the most. Determining the most and least mainstream artists can be solved with a single command each to the Dataframe. Determining the most influential artists requires that, for every artist, the listening average to be recalculated with that artist excluded. Subtracting the overall listener average from this number yields a positive or negative number signifying how much the overall average would have changed if that artist had not been included. For example, if a user listened to Kendrick Lamar – the most popular artist at the time

of writing – once and Green Day – a popular artist, but much less so – 30-40 times, Kendrick Lamar would be shown as the most mainstream artist, but Green Day would have a further negative value here, meaning if the user had not listened to any Green Day, the number would have been lower than if they had instead not listened to any Kendrick Lamar.

## 6.4 DESIGN DECISIONS

# CHAPTER 7

## CONCLUSION

## REFERENCES

- [1] Olatunde Adediji. *Full-Stack Flask and React: Learn, code, and deploy powerful web applications with Flask 2 and React 18*. en. Google-Books-ID: xGHWEEAAQBAJ. Packt Publishing Ltd, Oct. 2023. ISBN: 978-1-80323-660-5 (pages 12–13, 20).
- [2] Tim Faulkes. *SQL vs. NoSQL: Which Database is Right for Your Application?* en. URL: <https://aerospike.com/blog/sql-vs-nosql/> (visited on 12/01/2025) (page 23).
- [3] J. J. Geewax. *API Design Patterns*. en. Google-Books-ID: XWU0EAAQBAJ. Simon and Schuster, July 2021. ISBN: 978-1-61729-585-0 (page 4).
- [4] Mihaela Roxana Ghidersa. *Software Architecture for Web Developers: An introductory guide for developers striving to take the first steps toward software architecture or just looking to grow as professionals*. en. Google-Books-ID: 6E2OEAAQBAJ. Packt Publishing Ltd, Oct. 2022. ISBN: 978-1-80323-161-7 (pages 14–15).
- [5] Miguel Grinberg. *Flask Web Development*. en. 2nd. Google-Books-ID: cVIPDwAAQBAJ. "O'Reilly Media, Inc.", Mar. 2018. ISBN: 978-1-4919-9169-5.
- [6] Michael J. Hernandez. *Database Design for Mere Mortals: A Hands-On Guide to Relational Database Design*. en. Third. Google-Books-ID: MVApXsrcHiAC. Addison-Wesley, July 2021. ISBN: 978-0-13-312227-5 (page 22).
- [7] Arnaud Lauret. *The Design of Web APIs, Second Edition*. en. 2nd. Google-Books-ID: CRxEQAAQBAJ. Simon and Schuster, July 2025. ISBN: 978-1-63343-814-9 (page 5).
- [8] Ryan Mitchell. *Web Scraping with Python: Collecting Data from the Modern Web*. en. Google-Books-ID: 7z\_fCQAAQBAJ. "O'Reilly Media, Inc.", June 2015. ISBN: 978-1-4919-1025-2 (pages 8, 10–11, 30).
- [9] *Overview of HTTP - HTTP | MDN*. en-US. July 2025. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview> (visited on 11/23/2025) (page 13).
- [10] Yangshun Tay. *In-Depth Overview | Flux*. June 2020. URL: <https://web.archive.org/web/20200617175837/https://facebook.github.io/flux/docs/in-depth-overview/> (visited on 11/19/2025) (page 20).

- [11] William. *Web Application Architecture: The Latest Guide (2025 AI Update)*. en-US. July 2025. URL: <https://www.clickittech.com/software-development/web-application-architecture/> (visited on 11/15/2025) (page 15).

